ATC Developer Guide

Advanced Trigonometry Calculator Developer Guide

This guide gives a high-level view of the Advanced Trigonometry Calculator (ATC) codebase. It is intentionally factual and cautious; source code remains the authority for exact behavior.

For contribution recipes, checklists and practical "how to add" notes, see `docs/en/Developer_Reference.md`.

Project Purpose

ATC is a C++ Windows command-line mathematical application. It evaluates text-based expressions and documented commands in a console environment.

The project supports numerical workflows involving arithmetic, trigonometry, complex numbers, matrices, statistics, polynomial tools, equation solving, variables, and configurable precision.

ATC should not be described as having broad symbolic-algebra capabilities that are not documented in the repository.

Source Structure

The main source directory is:

```
Advanced Trigonometry Calculator/
```

Important source areas include:

- `main.cpp`: startup, precision-mode dispatch, top-level console flow;
- `auto_complete.cpp`: interactive line editing, Tab completion vocabulary, history navigation and command/function suggestions;
- `main_aux_processor.cpp` and `main_processor.cpp`: command and expression routing around the main calculation flow;
- `processing_core.cpp`: expression processing, arithmetic solving, and function processing;
- `commands.cpp`: many user-facing command handlers;
- `equation_solver.cpp` and `equations_system_solver.cpp`: equation-solving paths;
- `solver.cpp`: numerical solver functionality;
- `polynomial_arithmetic.cpp`: polynomial simplification and related helpers;
- `function_study.cpp`: function study workflow;
- `dynamic_allocations.cpp`: dynamic allocation helpers and tracking support;
- `data_processing_core.cpp`: data/settings/helper processing used across multiple features;
- mathematical modules such as `trigonometry.cpp`, `hyperbolic.cpp`, `logarithmic.cpp`, `statistics.cpp`, `digital_signal_processing.cpp`, and related calculation files.

Project-wide declarations are concentrated in `stdafx.h` and helper headers such as `precision_types.h`, `safe_chararray.h`, and `safe_index.h`.

Command Processing Flow

For a visual overview, see the "Module Architecture and Execution Flow" section in `docs/Architecture.md`.

At a high level:

1. `main.cpp` initializes ATC paths/settings and selects `double` or Boost `mp_float` based on persisted precision settings. 2. `mainType<T>()` enters the typed runtime flow. 3. User input or command-line arguments are routed into the main processing functions. 4. `main_core<T>()` and `main_sub_core<T>()` coordinate commands, variables, output, and expression evaluation. 5. Mathematical expressions pass through `initialProcessor<T>()`, `arithSolver<T>()`, and `functionProcessor<T>()` as needed.

The exact route depends on command type, current settings, and whether ATC is running interactively or through redirected/command-line input.

Parser and Expression Evaluation Overview

Expression processing is implemented in the existing C++ parser flow rather than through an external parser library.

The processing core handles:

- arithmetic operators;
- function calls;
- constants such as `pi`, `e`, and infinity markers;
- complex values;
- matrix expressions;
- implicit multiplication forms;
- precision-dependent result formatting.

ATC also uses documented syntax conventions such as _ for negative values in some internal/output forms.

Interactive Prompt and Autocomplete

The interactive prompt uses a custom console line editor instead of plain `gets_s(...)`. It supports:

- Tab completion while the user is typing;
- repeated Tab cycling when more than one completion is possible;
- shortest closest-match ordering for command/function suggestions;
- Up/Down history navigation using `history.txt`;
- Left/Right, Home, End, Delete and Backspace editing;
- suggestions for documented mathematical functions, commands, aliases and

user functions stored under the ATC `User functions` folder.

The autocomplete path should remain lightweight because it runs during normal typing. Suggestions are cached for the current line so repeated Tab presses do not rebuild the full list unnecessarily.

Equation Solver Overview

ATC supports several equation-related paths:

- quadratic equation command;
- systems of equations command;
- coefficient-form polynomial solving;
- supported textual polynomial normalization;
- selected rational-cancellation fast paths;
- numerical solver workflows through `solver(...)`.

The equation-solver documentation and tests currently validate simple textual polynomials, coefficient lists, high-degree polynomial examples, rational cancellation, and symbolic constants such as `pi`, `e`, and complex `pii`.

Unsupported expressions should remain on existing fallback paths or be rejected clearly rather than being treated as solved.

Precision Mode Overview

ATC 2.1.9 can run using:

- `double`
- Boost `mp_float`

The persisted precision setting is stored in:

```
%USERPROFILE%\Pictures\Advanced Trigonometry Calculator\higherPrecision.txt
```

Startup reads the persisted value and dispatches the main template flow to the selected numeric type. Some commands also support temporary high-precision evaluation through the documented `maxprec` prefix.

Memory Considerations

The project uses custom dynamic allocation helpers in `dynamic_allocations.cpp`. Recent 2.1.8 work reduced unnecessary allocation in several areas and improved release behavior for dynamic arrays.

When changing memory-sensitive code:

- use existing allocation/deletion helpers where appropriate;
- avoid large fixed-size buffers when the input size is known;
- preserve null termination for dynamic character arrays;
- release temporary arrays on all return paths;
- run memory stress tests when practical.

Test Infrastructure

Automated tests live in `tests/`.

Main scripts:

- `run-atc-regression.ps1`: broad regression runner;
- `run-atc-memory-stress.ps1`: repeated memory-sensitive command runner;
- `run-atc-script-benchmark.ps1`: script throughput and memory benchmark;
- `run-atc-isolated-coverage.ps1`: helper for isolated coverage scenarios.

Historical validation recorded before the final 2.1.9 candidate:

These results do not approve the current candidate. Final validation must identify the source snapshot and executable hashes.

```
Summary: 377 passed, 0 failed
```

Current isolated coverage result:

```
Summary: 68 passed, 0 failed
```

See `docs/Testing.md` for commands and coverage details.

Practical Developer Reference

The dedicated Developer Reference covers:

- how to add a new command;
- how to add a mathematical function;
- how to add a guided module;
- how to add regression tests;
- documentation expectations;

- parser and solver regression risks;
- Windows compatibility considerations;
- output-stability rules;
- pre-commit and pre-release checklist.

Use it together with this guide and the architecture overview before changing parser, solver, console or persistence behavior.